



PROGRAMMING WITH C++

UNIT 3

STRING, FILES AND POINTERS



Devang Patel Institute of Advance Technology and Research

Outlines

- String
- String Operations (Comparing, Assignment, Concatenation, Swapping, substring) and String Characteristics
- Numeric Conversions of string and string_view
- Pointer to object and this pointer
- Stream classes for files
- Sequential I/O operations
- Error handling during file operation

String

- In C++, a string is a sequence of characters used to represent text. C++ provides two main ways to handle strings:
 - **C-style strings (character arrays)**
 - **std::string class from the Standard Template Library (STL)**

C-Style Strings (Character Arrays)

- These are simple arrays of characters, ending with a null character ('\\0') to indicate the end of the string.
- Key Points:
 - Requires manual handling of sizes.
 - Prone to buffer overflow if not careful.
 - Limited built-in operations compared to `std::string`.

Example

```
#include <iostream>
using namespace std;

int main() {
    char cstr[] = "Hello, C-style!";
    cout << cstr << endl;

    // Accessing characters
    cout << "First character: " << cstr[0] << endl;

    return 0;
}
```

Output:

```
Hello, C-style!
First character: H
```

std::string (C++ Standard Library)

- The std::string class is more versatile and safer than C-style strings. It includes many helpful member functions.
- **C-Style vs. std::string**

Feature	C-Style Strings	std::string
Memory Management	Manual (prone to errors)	Automatic & safer
Functionality	Limited (C standard library)	Rich built-in methods
Safety	Low (buffer overflows)	High (bounds checking)
Concatenation	strcat(), strcpy()	+= or .append()
Ease of Use	Complex	Simple & intuitive

Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str = "Hello, C++ strings!";

    // Output the string
    cout << str << endl;

    // String concatenation
    str += " They're powerful.";
    cout << str << endl;

    // Access individual characters
    cout << "First character: " << str[0] << endl;

    // String length
    cout << "Length: " << str.length() << endl;

    // Substring
    cout << "Substring: " << str.substr(7, 5) << endl;

    // Find a word
    size_t pos = str.find("powerful");
    if (pos != string::npos) {
        cout << "'powerful' found at position: " << pos << endl;
    }

    return 0;
}
```

Output:

```
Hello, C++ strings!
Hello, C++ strings! They're powerful.
First character: H
Length: 37
Substring: C++ s
'powerful' found at position: 28
```

C++ String Operations

- C++ provides powerful string operations via the `std::string` class, which makes manipulating text straightforward and intuitive.

Operation	Operator/Method	Description
Compare	<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code>compare()</code>	Compare strings lexicographically.
Assign	<code>=</code> , <code>assign()</code>	Assign one string to another.
Concatenate	<code>+</code> , <code>+=</code> , <code>append()</code>	Combine strings together.
Swap	<code>swap()</code>	Swap two strings without extra overhead.
Substring	<code>substr(pos, len)</code>	Extract part of a string.

String Comparison Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str1 = "Apple";
    string str2 = "Banana";

    // Using comparison operators
    if (str1 == str2) {
        cout << "Strings are equal." << endl;
    } else if (str1 > str2) {
        cout << str1 << " is greater than " << str2 << endl;
    } else {
        cout << str1 << " is less than " << str2 << endl;
    }

    // Using compare() method
    int result = str1.compare(str2);
    if (result == 0) {
        cout << "Strings are equal." << endl;
    } else if (result > 0) {
        cout << str1 << " is greater than " << str2 << endl;
    } else {
        cout << str1 << " is less than " << str2 << endl;
    }

    return 0;
}
```

Output:

```
Apple is less than Banana
Apple is less than Banana
```

String Assignment Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str1 = "Hello";
    string str2;

    // Direct assignment
    str2 = str1;

    // Assign using assign()
    string str3;
    str3.assign("World");

    cout << "str1: " << str1 << endl;
    cout << "str2: " << str2 << endl;
    cout << "str3: " << str3 << endl;

    return 0;
}
```

Output:

```
str1: Hello
str2: Hello
str3: World
```

String Concatenation Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str1 = "Hello";
    string str2 = " World";

    // Using + operator
    string result = str1 + str2;
    cout << "Concatenated: " << result << endl;

    // Using += operator
    str1 += str2;
    cout << "Concatenated with +=: " << str1 << endl;

    // Using append()
    string str3 = "Welcome ";
    str3.append("to C++!");
    cout << "Using append(): " << str3 << endl;

    return 0;
}
```

Output:

```
Concatenated: Hello World
Concatenated with +=: Hello World
Using append(): Welcome to C++!
```

String Swapping Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str1 = "Apple";
    string str2 = "Orange";

    cout << "Before swap:\n";
    cout << "str1: " << str1 << ", str2: " << str2 << endl;

    // Swap using swap()
    str1.swap(str2);

    cout << "After swap:\n";
    cout << "str1: " << str1 << ", str2: " << str2 << endl;

    return 0;
}
```

Output:

```
Before swap:
str1: Apple, str2: Orange
After swap:
str1: Orange, str2: Apple
```

Substring Extraction Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str = "C++ Programming";

    // Extract substring (start index, length)
    string sub = str.substr(4, 11);
    cout << "Substring: " << sub << endl;

    // Get the last word
    string lastWord = str.substr(str.find("Programming"));
    cout << "Last word: " << lastWord << endl;

    return 0;
}
```

Output:

```
Substring: Programming
Last word: Programming
```

String Methods

Method	Usage	Description	Example Code	Expected Output
assign()	s.assign("Hello")	Assigns new content to the string.	string s; s.assign("Hello"); cout << s;	Hello
append()	s.append(" World")	Appends a string at the end.	string s="Hello"; s.append(" World");	Hello World
insert()	s.insert(5,"123")	Inserts substring at position.	string s="Hello"; s.insert(5,"123");	Hello123
replace()	s.replace(6,5,"C++")	Replaces part of the string.	string s="Hello World"; s.replace(6,5,"C++");	Hello C++
erase()	s.erase(5,6)	Erases part of the string.	string s="Hello World"; s.erase(5,6);	Hello
substr()	s.substr(6,5)	Extracts substring from the string.	string s="Hello World"; cout<<s.substr(6,5);	World
find()	s.find("World")	Finds the first occurrence of substring.	string s="Hello World"; cout<<s.find("World");	6
rfind()	s.rfind("Hello")	Finds the last occurrence of substring.	string s="Hello Hello"; cout<<s.rfind("Hello");	6

String Methods

Method	Usage	Description	Example Code	Expected Output
find_first_of())	s.find_first_of("aeiou")	Finds first vowel occurrence.	string s="hello"; cout<<s.find_first_of("aeiou");	1
find_last_of()	s.find_last_of("aeiou")	Finds last vowel occurrence.	string s="hello"; cout<<s.find_last_of("aeiou");	4
compare()	s1.compare(s2)	Compares two strings.	string s1="abc", s2="abc"; cout<<s1.compare(s2);	0
size()/length() ()	s.size()	Returns string length.	string s="Hello"; cout<<s.size();	5
capacity()	s.capacity()	Returns the storage capacity.	string s="Hello"; cout<<s.capacity();	Varies
empty()	s.empty()	Checks if the string is empty.	string s=""; cout<<s.empty();	1 (true)
clear()	s.clear()	Clears the string content.	string s="Hello"; s.clear(); cout<<s;	<i>empty</i>
at()	s.at(1)	Accesses character at index.	string s="Hello"; cout<<s.at(1);	e
front()/back() ()	s.front(), s.back()	Returns first/last character.	string s="Hello"; cout<<s.front()<<s.back();	Ho
push_back()	s.push_back('!')	Appends a character.	string s="Hi"; s.push_back('!'); cout<<s;	Hi!
pop_back()	s.pop_back()	Removes the last character.	string s="Hi!"; s.pop_back(); cout<<s;	Hi
resize()	s.resize(3)	Resizes the string.	string s="Hello"; s.resize(3); cout<<s;	Hel
swap()	s1.swap(s2)	Swaps two strings.	string s1="Hi", s2="Bye"; s1.swap(s2);	s1:Bye s2:Hi

C++ String Characteristics

- The `std::string` class in C++ is a versatile, dynamic, and memory-safe way to handle text.
 - **Dynamic Size (Resizable)**
 - **Null-Terminated (like C-style)**
 - **Length and Capacity Management**
 - **Memory Safety**
 - **Immutable Individual Characters**

Dynamic Size (Resizable)

- Strings can grow or shrink dynamically without manual memory management.
- Internally, memory is automatically reallocated when needed.

Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str = "Hello";
    str += ", World!"; // Resized automatically
    cout << str << endl; // Output: Hello, World!
    return 0;
}
```

Output:

Hello, World!

Null-Terminated (like C-style)

- Internally, `std::string` is null-terminated (`'\0'`), ensuring compatibility with C-style functions when needed.
- However, you don't need to manage the `'\0'` yourself.

Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str = "Hello";
    cout << "C-style string: " << str.c_str() << endl; // c_str()
    returns a null-terminated C-string
    return 0;
}
```

Output:

```
C-style string: Hello
```

Length and Capacity Management

- `length()` and `size()` return the number of characters.
- `capacity()` shows allocated memory, which may be greater than `length()`.
- `resize()` allows manual size adjustments.

Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str = "Hello";
    cout << "Length: " << str.length() << endl;
    cout << "Capacity: " << str.capacity() << endl;

    str.resize(10, '!');
    cout << "After resize: " << str << endl; // Output: Hello!!!!
    return 0;
}
```

Output:

```
Length: 5
Capacity: 15
After resize: Hello!!!!
```

Memory Safety

- Bounds-checked when using `.at()` (throws `std::out_of_range` if invalid).
- Unchecked with `[]` (like arrays, may cause undefined behavior if out-of-bounds).

Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str = "SafeString";

    // Safe access
    cout << "Character at index 4 (safe): " << str.at(4) << endl;

    // Risky access
    cout << "Character at index 4 (unsafe): " << str[4] << endl;

    // Out-of-range example (commented to avoid crash)
    // cout << str.at(20); // Throws an exception

    return 0;
}
```

Output:

```
Character at index 4 (safe): S
Character at index 4 (unsafe): S
```

Immutable Individual Characters

- Strings are mutable as a whole but individual characters can be directly modified.
- Cannot assign strings using single quotes without conversion.

Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str = "Mutable";
    str[0] = 'I';
    cout << str << endl; // Output: Iutable
    return 0;
}
```

Output:

Iutable

Numeric Conversions of string

- C++ provides several methods to convert between strings and numeric types (e.g., int, float, double) using built-in functions from the Standard Library.
- The `std::stoi`, `std::stol`, `std::stoll`, `std::stof`, `std::stod`, `std::stold` functions can convert strings into various numeric types.

Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string intStr = "12345";
    string floatStr = "45.67";

    // Integer conversions
    int i = stoi(intStr);
    long l = stol(intStr);
    long long ll = stoll(intStr);

    // Floating-point conversions
    float f = stof(floatStr);
    double d = stod(floatStr);
    long double ld = stold(floatStr);

    // Output results
```

```
    cout << "int: " << i << ", long: " << l << ", long long: " << ll << endl;
    cout << "float: " << f << ", double: " << d << ", long double: " << ld <<
    endl;

    return 0;
}
```

Output:

```
int: 12345, long: 12345, long long: 12345
float: 45.67, double: 45.67, long double: 45.67
```

Converting Numeric Types to `std::string`

- The `std::to_string()` function easily converts numbers into strings.

Example

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int i = 42;
    float f = 3.14159;
    double d = 2.71828;

    // Convert numbers to strings
    string intStr = to_string(i);
    string floatStr = to_string(f);
    string doubleStr = to_string(d);

    // Output results
    cout << "intStr: " << intStr << endl;
    cout << "floatStr: " << floatStr << endl;
    cout << "doubleStr: " << doubleStr << endl;

    return 0;
}
```

Output:

```
intStr: 42
floatStr: 3.141590
doubleStr: 2.718280
```

Numeric Conversions with `std::string_view`

- The `std::string_view` class (introduced in C++17) provides a lightweight, read-only view of a string.
- It doesn't directly support numeric conversions, but you can convert a `string_view` to a `string` easily and then use the existing functions.
- Why the Extra Step?
 - `std::string_view` is a lightweight view and does not provide numeric conversion functions directly.
 - The workaround: `string_view -> string -> numeric`.

Example

```
#include <iostream>
#include <string>
#include <string_view>
using namespace std;

int main() {
    string_view sv = "12345";

    // Convert string_view to string, then to int
    int number = stoi(string(sv));

    cout << "Converted number: " << number << endl;

    return 0;
}
```

Output:

```
Converted number: 12345
```

Basics of Pointers

- In C++, pointers are variables that store the memory address of another variable.
- They are fundamental to understanding memory management, dynamic allocation, and efficient data structures like linked lists and trees.

Example

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int* ptr = &x; // ptr holds the address of x

    cout << "Value of x: " << x << endl;      // Value
    cout << "Address of x: " << &x << endl;    // Address of x
    cout << "Pointer ptr: " << ptr << endl;    // Address stored in ptr
    cout << "Value at ptr: " << *ptr << endl;  // Dereference to get value

    return 0;
}
```

Output:

```
Value of x: 10
Address of x: 0x7ffc413621e4
Pointer ptr: 0x7ffc413621e4
Value at ptr: 10
```

Pointer Arithmetic Example

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int* ptr = arr;

    cout << "First element: " << *ptr << endl;
    ptr++; // Moves to the next element (4 bytes for int)
    cout << "Second element: " << *ptr << endl;

    return 0;
}
```

Output:

```
First element: 10
Second element: 20
```

Dynamic Memory Allocation Example

```
#include <iostream>
using namespace std;

int main() {
    // Dynamic memory allocation
    int* ptr = new int(100);

    cout << "Value: " << *ptr << endl;

    // Free the allocated memory
    delete ptr;

    return 0;
}
```

Output:

Value: 100

Pointer to Objects

- In C++, **pointers to objects** allow us to dynamically allocate, access, and manipulate objects in memory.
- This is particularly useful when working with **dynamic polymorphism, dynamic memory allocation, and object-oriented programming**.

Example

```
#include <iostream>
using namespace std;

class Person {
public:
    string name;
    int age;

    void display() {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};

int main() {
    Person person1;
    person1.name="Alice";
    person1.age=25;

    Person* ptr = &person1;

    // Access members using pointer
    cout << "Using pointer: " << ptr->name << ", " << ptr->age << endl;

    // Call member function using pointer
    ptr->display();

    return 0;
}
```

Output:

```
Using pointer: Alice, 25
Name: Alice, Age: 25
```

Dynamic Allocation of Objects

```
#include <iostream>
using namespace std;

class Car {
public:
    string brand;
    int year;

    void display() {
        cout << "Brand: " << brand << ", Year: " << year << endl;
    }
};

int main() {
    // Dynamically allocate object
    Car* carPtr = new Car;

    // Assign values via pointer
    carPtr->brand = "Toyota";
    carPtr->year = 2024;

    // Display object details
    carPtr->display();

    // Deallocate memory
    delete carPtr;

    return 0;
}
```

Output:

Brand: Toyota, Year: 2024

Array of Objects with Pointers Example

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int age;

    void display() {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};

int main() {
    // Allocate an array of objects
    Student* students = new Student[3];

    // Assign values
    students[0].name = "John"; students[0].age = 20;
    students[1].name = "Jane"; students[1].age = 21;
```

```
    students[2].name = "Jack"; students[2].age = 22;

    // Display all students
    for (int i = 0; i < 3; i++) {
        students[i].display();
    }

    // Deallocate memory
    delete[] students;

    return 0;
}
```

Output:

```
Name: John, Age: 20
Name: Jane, Age: 21
Name: Jack, Age: 22
```

this Pointer

- In C++, the this pointer is an implicit, local pointer available to all non-static member functions of a class.
- It points to the calling object, i.e., the instance of the class for which the member function was called.
- **Key Points:**
 - **this is an implicit pointer:** You don't need to declare it; it's automatically available within all non-static member functions.
 - **Points to the current object:** It stores the address of the object that invoked the member function.
 - **Cannot be modified:** The this pointer is a const pointer, meaning it cannot be reassigned to point to another object.
 - **Used in method chaining:** Returning *this allows you to chain method calls on the same object.

Example

```
#include <iostream>
using namespace std;

class Person {
private:
    string name;
    int age;

public:
    // Constructor with parameter names matching member
    // variables
    Person(string name, int age) {
        this->name = name; // Resolving name shadowing
        this->age = age;
    }

    void display() {
        cout << "Name: " << this->name << ", Age: " << this->age <<
        endl;
    }
};

int main() {
    Person p1("Alice", 25);
    p1.display();

    return 0;
}
```

Output:

Name: Alice, Age: 25

this Pointer in Method Chaining

- The this pointer can be returned from a member function, allowing method chaining.

Example

```
#include <iostream>
using namespace std;

class Calculator {
private:
    int value;

public:
    Calculator() : value(0) {}

    // Method to add a number
    Calculator* add(int num) {
        this->value += num;
        return this;
    }

    // Method to multiply
    Calculator* multiply(int num) {
        this->value *= num;
        return this;
    }

    // Display the result
    void display() {
        cout << "Result: " << this->value << endl;
    }
};

int main() {
    Calculator calc;
    calc.add(10)->multiply(5)->display();

    return 0;
}
```

Output:

Result: 50

File Handling

- C++ file handling allows you to create, read, write, and manipulate files using the file stream classes provided by the `<fstream>` header.
- It enables programs to persist data by storing it in files, which can be accessed and modified later.
- File Handling Classes in C++The `<fstream>` header provides three main classes for file handling:
 - `ofstream` (output file stream): For writing to files.
 - `ifstream` (input file stream): For reading from files.
 - `fstream` (file stream): For both reading and writing.

File Handling Methods

- C++ provides several file handling methods to perform operations like opening, closing, reading, writing, checking errors, and more.
- These methods belong to the ifstream, ofstream, and fstream classes from the <fstream> library.

Method	Description
.open("filename", mode)	Opens a file in a specific mode
.close()	Closes the opened file
.is_open()	Checks if a file is successfully opened
.write((char*)data, size)	Writes binary data to a file
.read((char*)data, size)	Reads binary data from a file
.getline(variable, size)	Reads a line from the file
.eof()	Checks if the end of the file is reached
.fail()	Checks if a file operation has failed
.seekg(position)	Moves the read pointer to a specific position
.seekp(position)	Moves the write pointer to a specific position
.tellg()	Returns the current position of the read pointer
.tellp()	Returns the current position of the write pointer

File Opening Modes

Each file stream class supports different **file opening modes**:

Mode	Description
<code>ios::in</code>	Open for reading
<code>ios::out</code>	Open for writing (overwrites existing content)
<code>ios::app</code>	Append mode (data is added to the end)
<code>ios::ate</code>	Open and move to the end of the file
<code>ios::trunc</code>	Truncate file (delete content if file exists)
<code>ios::binary</code>	Open file in binary mode

Example:

```
fstream file("example.txt", ios::in | ios::out | ios::app);
```

Writing to a File

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream outFile("example.txt"); // Open file for writing

    if (outFile.is_open()) {
        outFile << "Hello, C++ file handling!\n";
        outFile << "This is a test.\n";
        outFile.close(); // Close the file
        cout << "Data written successfully!\n";
    } else {
        cout << "Failed to open the file.\n";
    }

    return 0;
}
```

Reading from a File

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream inFile("example.txt"); // Open file for reading
    string line;

    if (inFile.is_open()) {
        while (getline(inFile, line)) { // Read line by line
            cout << line << endl;
        }
        inFile.close(); // Close the file
    } else {
        cout << "Failed to open the file.\n";
    }

    return 0;
}
```

Reading & Writing (Both)

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream file("example.txt", ios::in | ios::out | ios::app);

    if (file.is_open()) {
        file << "Appending new line with fstream.\n";

        file.seekg(0); // Move the read pointer to the
beginning
        string line;
        while (getline(file, line)) {
            cout << line << endl;
        }

        file.close();
    } else {
        cout << "Failed to open the file.\n";
    }

    return 0;
}
```

Sequential I/O operations

- Sequential I/O operations in C++ involve reading and writing data to files in a continuous, linear order.
- This means processing the file from start to end, without skipping around or modifying existing content at specific positions.
- It is the most common way of handling files in applications like logging, configuration files, and data storage.

Operation	Class	Method Used
Write sequentially	ofstream	<<
Read sequentially (line by line)	ifstream	getline(file, line)
Read sequentially (word by word)	ifstream	file >> word
Write and Read sequentially	fstream	<<, getline()
Check end of file	ifstream	.eof()

Writing Data Sequentially (ofstream)

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream outFile("data.txt"); // Open file for writing

    if (outFile.is_open()) {
        outFile << "Line 1: Welcome to C++ File Handling\n";
        outFile << "Line 2: This is sequential writing\n";
        outFile << "Line 3: Data is written line by line\n";
        outFile.close(); // Close the file
        cout << "Data written successfully!" << endl;
    } else {
        cout << "Error opening file!" << endl;
    }

    return 0;
}
```

Reading Data Sequentially (ifstream)

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream inFile("data.txt"); // Open file for reading
    string line;

    if (inFile.is_open()) {
        while (getline(inFile, line)) { // Read line by line
            cout << line << endl;
        }
        inFile.close(); // Close the file
    } else {
        cout << "Error opening file!" << endl;
    }

    return 0;
}
```

Writing & Reading Sequentially (fstream)

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream file("data.txt", ios::in | ios::out | ios::app); // Open for
    read/write in append mode

    if (file.is_open()) {
        file << "Line 4: Appending new content\n"; // Writing sequentially
        file.seekg(0); // Move to beginning for reading

        string line;
        cout << "Reading file sequentially:" << endl;
        while (getline(file, line)) { // Reading sequentially
            cout << line << endl;
        }

        file.close();
    } else {
        cout << "Error opening file!" << endl;
    }

    return 0;
}
```

Checking End of File (eof())

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("data.txt");
    string word;

    if (file.is_open()) {
        while (!file.eof()) { // Read word by word
            file >> word;
            cout << word << " ";
        }
        file.close();
    } else {
        cout << "Error opening file!" << endl;
    }

    return 0;
}
```

Error handling during file operation

- When working with files in C++, errors can occur due to various reasons, such as:
- File not found
- No read/write permissions
- Disk full
- File corruption
- To handle such errors gracefully, we use error-checking methods like `.fail()`, `.eof()`, and `.bad()` from the `<fstream>` library.

Checking if a File Opened Successfully (is_open())

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("nonexistent.txt"); // Trying to open a file
    that does not exist

    if (!file.is_open()) {
        cout << "Error: File could not be opened!" << endl;
        return 1; // Exit the program with an error code
    }

    cout << "File opened successfully!" << endl;
    file.close();
    return 0;
}
```

Handling File Read/Write Errors (fail())

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("data.txt");

    if (file.fail()) {
        cout << "Error: Unable to open file!" << endl;
        return 1;
    }

    string data;
    file >> data; // Trying to read data
    if (file.fail()) {
        cout << "Error: Failed to read from file!" << endl;
    } else {
        cout << "Data read: " << data << endl;
    }

    file.close();
    return 0;
}
```

Checking for Critical Errors (bad())

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("data.txt");

    if (file.bad()) {
        cout << "Critical error: File is corrupted or unreadable!"
        << endl;
        return 1;
    }

    cout << "File is accessible!" << endl;
    file.close();
    return 0;
}
```

Clearing Errors (clear())

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("data.txt");

    if (!file) {
        cout << "Error opening file!" << endl;
        file.clear(); // Reset error flags
    }

    file.close();
    return 0;
}
```